

Btrfs и btrbk: лёгкий и быстрый инкрементальный бэкап сервера и домашнего ПК

 habr.com/ru/companies/ruvds/articles/1035534

May 15, 2026

Хабр



**Btrfs и btrbk: лёгкий и быстрый
инкрементальный бэкап сервера
и домашнего ПК**

RUVDS.com



Btrfs + btrbk — это революция в мире бэкапа

Старая сисадминская пословица гласит: люди делятся на две категории — на тех, кто уже делает резервные копии, и тех, кто будет их делать.

Многие админы знакомы с этой ужасной сценой: 3 часа ночи, данные потеряны, и ты в панике, в холодном поту лихорадочно гуглишь, как спасти данные или ищешь телефон конторы по восстановлению данных и ужасаешься их ценам. Избежать таких факапов помогает полезная привычка — вовремя и правильно делать бакап.

В этой статье я покажу свой рабочий способ резервного копирования: файловая система Btrfs плюс утилита [btrbk](#). Это самый лёгкий и быстрый инкрементальный бакап и для сервера, и для домашнего ПК, который я знаю. Настройка занимает десять минут, ежедневный бакап — считанные минуты.

Btrfs + btrbk — это революция в мире бакапа. Ещё никогда не было так просто и быстро создавать дифференциальные резервные копии. Никаких лицензий, подписок, платных программ — всё бесплатно и встроено в ядро Linux.

Как я делал инкрементальные бакапы раньше

Много лет домашний каталог сервера с пользовательскими данными я бакапил утилитой [dar](#) (Disk ARchive) французского разработчика Дени Корбена (Denis Corbin). Это две программы: архиватор `dar`, который создаёт полные и дифференциальные архивы, и `dar_manager` — менеджер индекса архивов, который ведёт базу всех архивов и умеет восстанавливать файлы на нужную дату. Инструмент отличный: полные, дифференциальные и инкрементальные архивы, нарезка на тома нужного размера, выборочная архивация, сжатие разными алгоритмами, в том числе очень быстрым для распаковки `zstd`.

Но любой файловый архиватор работает на уровне файлов. `dar` честно сравнивает файл с архивированным по времени и размеру — если файл изменился, в дифференциальный архив он уходит целиком. На каталоге `/home` это означало:

- переименовал большую папку — она уезжает в архив заново, целиком;
- базы данных перебакапивались заново всегда — стоило измениться хоть одному байту внутри файла БД, и многогигабайтный файл целиком попадал в «дифференциальную» копию;
- каждый прогон — это полный обход дерева всех каталогов: чтобы понять, что изменилось, `dar` обязан обойти все пользовательские каталоги и сверить `stat()` каждого файла с архивированными до этого — на миллионах файлов это десятки минут только на сканирование, ещё до того, как скопирован первый байт.

Полный обход дерева при каждом запуске — минус всех пофайловых бакаперов без исключения: `dar`, `rsync`, `tar`, коммерческих программ. Ни одна из них не знает, что поменялось, пока не обойдёт всё дерево целиком.

В итоге «дифференциальный» бакап на живых данных по объёму и времени мало отличался от полного. Это не проблема `dar` — это недостаток файлового подхода. Для ФС без поддержки снимков (`ext4`, `XFS`, `FAT32`, `exFAT`, `NTFS` и т. п.) `dar` + `dar_manager` до сих пор остаётся хорошим решением. Тем более что `dar` — кроссплатформенный (Linux, Unix-подобные, Windows).

После `dar` я довольно долго бакапил `/home` программой `rsync`. Это прекрасная, заслуженная утилита, и плюсов у неё много: она есть в любом дистрибутиве и работает с любой файловой системой локально и через SSH, своим дельта-алгоритмом копирует только изменившуюся часть файла, умеет делать «снимки для бедных» на жёстких ссылках (`--link-dest`, как в `rsnapshot`, в бакапе это позволяет сослаться на файл из предыдущей копии без копирования), сохраняет права и сжимает на лету и т.д. и т.п. Но до скорости и удобства `btrbk` ей далеко:

- **Нет атомарности.** `rsync` копирует файлы по одному и идёт по дереву минутами, а то и часами. Пока обход идёт, файлы меняются — на выходе «размазанная» во времени копия, не соответствующая ни одному реальному состоянию системы. Чтобы копия была целостной, перед запуском приходится останавливать все активные процессы, которые пишат в `/home` (демоны, СУБД). Снимок `Btrfs` атомарен и мгновенен — останавливать почти ничего не нужно.
- **Полный обход дерева.** Та самая беда пофайлового подхода, о которой я писал выше про `dar`: `rsync` обязан `stat()` каждый файл, прежде чем поймёт, что копировать. На больших каталогах сканирование само по себе занимает десятки минут. `btrbk` не сканирует ничего.
- **Переименование = повторное копирование.** Передвинули или переименовали большой каталог — для `rsync` это «новые» файлы (в лучшем случае перелинковка через `--link-dest`, но всё равно с полным обходом). `Btrfs` передаёт несколько килобайт метаданных.
- **Нет точной точки во времени.** `rsync` синхронизирует «как успеется за время прохода», а `btrbk` фиксирует строгий срез на момент снимка.

Если файловая система умеет делать снимки, все эти минусы сразу исчезают: `Btrfs` работает не с файлами, а с блоками и деревом метаданных (об этом — ниже). Именно поэтому переход на `Btrfs` + `btrbk` я и называю революцией.

Был и третий сценарий — частный случай бакапа, обычно это про работу с большими файлами: на `Btrfs` их можно скопировать мгновенно через `copy-on-write` (`cp --reflink`). Копия создаётся за доли секунды и поначалу не занимает лишнего места. Тонкости и подводные камни этого режима я разбирал в отдельной статье — [«Что не так с Copy-on-Write под Linux при копировании»](#).

Когда я впервые забакапил с помощью Btrfs + btrbk огромную папку /home многопользовательского сервера — 1.5ТБ данных всего за две минуты, я ощутил полный восторг и эйфорию. Это было похоже на чудо. Тот же дифференциальный бакап этой папки через drg занимал у меня 6–8 часов. Шесть часов против двух минут! После такого обратно на файловый бакап не возвращаются.

► Новичкам. Зачем вообще делать бакап?

Полный, инкрементальный и дифференциальный бакап

Полный — копируется всё, целиком. Это база для других типов бакапа.

Инкрементальный — каждый прогон сохраняет только то, что изменилось с момента предыдущего бакапа любого типа (полного или прошлого инкремента).

Пн: ПОЛНЫЙ

Вт: инкремент = изменения с Пн

Ср: инкремент = изменения с Вт

Чт: инкремент = изменения с Ср

Объяснить с

- **Плюс:** каждый прогон минимальный по объёму и времени (только свежая дельта).
- **Минус:** для восстановления на четверг нужны полный + ВСЕ инкременты по цепочке по порядку. Потерялся/побился один кусок в середине — всё, что после него, не восстановить. Восстановление медленное (накатывать много слоёв).

Дифференциальный — каждый прогон сохраняет всё, что изменилось с момента последнего полного бакапа (а не с прошлого прогона).

Пн: ПОЛНЫЙ

Вт: дифф = изменения с Пн

Ср: дифф = изменения с Пн (вторник + среда)

Чт: дифф = изменения с Пн (вторник + среда + четверг)

Объяснить с

- **Плюс:** для восстановления нужны только полный + последний дифф — всего две части. Быстро и надёжно, нет длинной хрупкой цепочки.
- **Минус:** каждый следующий дифф толще и дольше предыдущего — каждый раз заново тащит всё, что накопилось с полного бакапа.

Связка Btrfs + btrbk даёт фактически полную копию на нужный момент времени ценой записи инкрементальных изменений на диск. То есть очень дёшево.

Сколько копий нужно: правило 3-2-1

Есть проверенное временем правило 3-2-1. Его сформулировал фотограф Питер Крог (Peter Krogh) в книге «*The DAM Book: Digital Asset Management for Photographers*», а позже его подхватил американский US-CERT в публикации [Data Backup Options](#).

- 3 копии данных,
- на 2 разных носителях,
- 1 из которых — вне помещения.

На практике: первая копия — на рабочей машине («удалил не ту папку»), вторая — на отдельном носителе, физически отключённом от неё, но в том же помещении («диск полетел»), третья — на удалённой площадке (ДЦ сгорел или сервер украли/конфисковали). Btrfs и btrbk закрывают все три сценария одним инструментом.

Чем Btrfs принципиально лучше «обычного» инкрементального бакапа

Классический инкрементальный бакап (rsync, tar, большинство коммерческих агентов) работает на уровне файлов: он сравнивает, что изменилось, и копирует дельту. Звучит хорошо, пока не появляется большой диск.

Представьте корпоративное хранилище или сервер удалённых профилей: у каждого пользователя папка на несколько гигабайт. Пользователь переименовал каталог размером 100 ГБ. Для файлового бакапа это 100 ГБ новых данных: по старому пути файлов больше нет, по новому — появились «новые». Вся дельта едет на бакап заново.

Btrfs работает не с файлами, а с блоками и деревом метаданных. Переименование каталога — это изменение нескольких килобайт метаданных. При инкрементальной передаче между снимками Btrfs отправит ровно эти несколько килобайт, а не 100 ГБ. То же самое с перемещением, дедупликацией и сору-он-write копиями. Именно поэтому на больших и «живых» данных Btrfs-бакап выигрывает у файлового на порядки.

Из-за этой простоты бакапа и снимков я со временем перевёл на Btrfs вообще все свои разделы: на моих серверах на ней и /boot, и /, и /home. Btrfs уже давно в основном ядре Linux (с 2009 года), её [дисковый формат стабилен](#) и меняться не

планирует, а ключевые вещи — снимки, `send/receive`, сжатие, контрольные суммы, RAID0/1/10 — помечены как стабильные. Единственное заметное исключение — встроенный RAID5/6, его в продакшене пока не используют. По умолчанию корень на Btrfs ставят openSUSE и SUSE Linux Enterprise (с 2014 года) и Fedora Workstation (с Fedora 33, 2020). А Meta (бывшая Facebook) держит на Btrfs миллионы серверов — все ОС и дата-тома. Всё это давно работает стабильно и чётко.

Снапшоты Btrfs: мгновенные и почти бесплатные

Снимок (снапшот) в Btrfs — это новый корень дерева подтома. Благодаря `copy-on-write` он не копирует данные: он фиксирует текущее состояние, а расходиться с оригиналом начинает только по мере изменений. Поэтому снимок создаётся мгновенно и почти не занимает места в момент создания.

Пара слов про имена. Подтома и снимки по сложившейся конвенции именуют с «собачки» — `@`, `@home`, `@snapshots` (в моих скриптах ниже снимки тоже идут как `@дата_метка`). Плоскую раскладку `@/@home` популяризировал установщик Ubuntu, отсюда её переняли openSUSE Leap, Manjaro и другие; а сам символ `@` перекочевал из ZFS, где снимок записывается как `пул/датасет@имя`. Каталог `.snapshots` для хранения снимков первым ввёл [snapper](#) из openSUSE — отсюда это имя и разошлось по экосистеме. `btrfs` по умолчанию кладёт снимки в `_btrfs_snap`; я в конфиге переназначаю это на привычный `.snapshots`. У меня, к слову, `/` и `/home` — вообще разные btrfs-тома, и в каждом свой `.snapshots`.

Раз он мгновенный — его можно делать на лету, прямо на работающей системе, не останавливая сервисы. Но если есть выбор — делайте бакап ночью. Что именно Btrfs при этом гарантирует, а что нет:

► Делает ли Btrfs `fsync` при снапшоте? Разбираем по официальной документации. Практический вывод: для файлов и большинства сервисов хватает обычного снимка на лету. Для баз данных и демонов, которые активно пишут на ФС и держат состояние в своей памяти, перед снимком лучше:

- на мгновение приостановить базу данных или активно пишущий демон (многие СУБД умеют режим `backup/freeze`) — снимок мгновенный, так что пауза занимает доли секунды;
- либо снимать ночью, когда сервис простаивает.

Мой способ: каталог `.snapshots` и два скрипта

В корне каждой файловой системы я создаю каталог `.snapshots` и кладу туда два маленьких скрипта — создать снимок и удалить снимок. Намеренно держу их

максимально простыми, без обвязки: их задача — делать именованные снимки с датой перед обновлением системы и сразу после него.

`create_snapshot.sh` — создать снимок с меткой времени и произвольным суффиксом:

```
#!/bin/bash

# снимок корня /; для /home замените / на /home и запускайте из /home/.snapshots
btrfs subvolume snapshot -r / @$ (date +"%Y%m%d_%H:%M")_$1
Объяснить с
```

`del_snapshot.sh` — удалить снимок по имени:

```
#!/bin/bash

btrfs subvolume delete $1
Объяснить с
```

Запускаю их из каталога `.snapshots`, передавая первым аргументом осмысленный суффикс — короткую метку, по которой через месяц будет понятно, что это за снимок и зачем он создан (`before_update`, `after_update`, `before_kernel`, `before_pg_upgrade`). Снимок получает имя вида `@20260515_10:30_before_update` — дата, время и причина видны прямо в `ls`. Типичный сценарий вокруг апдейта:

```
cd /.snapshots
./create_snapshot.sh before_update
emerge -uDN @world # или apt full-upgrade / dnf upgrade
./create_snapshot.sh after_update
Объяснить с
```

Создание мгновенно, так что снимок «до» ничего не стоит, а снимок «после» фиксирует уже рабочую систему. Если обновление сломало загрузку — откат это не переустановка, а возврат на снимок `before_update` (как — в разделе про `grub-btrfs` ниже). Снимки, которые больше не нужны, убираю `./del_snapshot.sh @20260515_10:30_before_update`.

Флаг `-r` делает снимок `read-only`. Так его нельзя случайно испортить, и он годится и как опорная точка для `btrfs send`, и для загрузки через `grub-btrfs`. Единственный плюс записываемого снимка — его можно прямо

примонтировать как рабочую систему при загрузке. Но для длительного восстановления это не годится: система поедет с подтома под другим именем, и пришлось бы править `/etc/fstab` (или менять подтом по умолчанию). Поэтому снимки держим `read-only`, а возвращаемся к ним через `btrfs subvolume snapshot` — см. раздел «Как восстановиться».

Автоматизация и ротация: btrbk

Делать снимки руками перед апдейтами — хорошо, но настоящий бэкап должен быть автоматическим и сам управлять своим жизненным циклом. Эту роль берёт на себя [btrbk](#) — инструмент Акселя Бурри (Digital Integrity GmbH). О нём я узнал от разработчиков Calculate Linux в их телеграм-чате поддержки — за наводку им отдельное спасибо. `btrbk` написан на Perl — это один скрипт без зависимостей, поэтому легко ставится даже там, где его нет в поддерживаемых пакетах дистрибутива; его описывают как «*backup tool for btrfs subvolumes... to create atomic snapshots and transfer them incrementally to your backup locations*».

`btrbk` делает три вещи:

1. Создаёт снимки по расписанию (ежедневно/еженедельно/...);
2. Инкрементально их сохраняет — на локальный диск или удалённый сервер по SSH;
3. Удаляет устаревшие снимки и копии согласно политике хранения.

То есть он держит актуальными ежедневные, еженедельные, ежемесячные и годовые копии и сам ротирует старые. Запускается из `cron` или `systemd`-таймера.

Мой btrbk.conf по шагам

Я взял `/etc/btrbk/btrbk.conf.example`, выкинул из него все демонстрационные блоки и оставил минимум.

Каталог снимков. По умолчанию `btrbk` кладёт снимки в `_btrbk_snap`. Я заменил на `.snapshots` — тот самый каталог из предыдущего раздела:

```
snapshot_dir          .snapshots
Объяснить с
```

Важная деталь из комментария в примере конфига: `btrbk` не создаёт каталог для снимков и упадёт, если его нет. Один раз сделайте `mkdir .snapshots` (и в корневой папке каждой нужной ФС).

Срок хранения и ротация копий

Я заменил пример `20d 10w *m` на:

```
target_preserve          7d 4w 3m 1y
Объяснить с
```

Читается так: на стороне бакапа хранить 7 дневных, 4 недельных, 3 месячных и 1 годовую копии. Всё, что старше и не попадает под эти слоты, `btrbk` удаляет автоматически. Локальные снимки-источники при этом живут по своему правилу `snapshot_preserve 14d` (14 дней) — это короткая «история на откат», полная глубина хранится уже на стороне бакапа.

Что бакапим. Бакап раздела `/home` на домашнем ПК:

```
# домашний раздел
volume /home
# резервные копии помещать сюда
target /mnt/backup/home
# снимать сам подтом
subvolume .
# имя подтома в снимке
snapshot_name home
Объяснить с
```

Для рабочего сервера я бакаплю не только `/home`, но и корень. Добавляется аналогичный блок:

```
# корневой раздел сервера
volume /
target /mnt/backup/root
subvolume .
snapshot_name root
Объяснить с
```

Бакап удалённого сервера

`btrbk run` запускается в двух местах. Для рабочего сервера одного бакапа на локальный диск мало — нужна копия в другом месте (третья копия по правилу 3-2-1). Это важный момент: `btrbk run` запускается на двух машинах (сервер и бакап-сервер) с разными конфигами.

- На рабочем сервере — для локального бакапа: сделать снимки и сложить их на локальный (желательно иногда отключаемый) диск. Это копии 1 и 2. Конфиг указан выше: блоки `volume /` и `volume /home` с локальными `target`.
- На бакап-сервере — чтобы он сам вытянул готовые снимки с рабочего сервера к себе. Это копия 3, в другом месте.

Почему за данными ходит бакап-сервер, а не рабочий толкает их наружу (это вопрос безопасности), и как настроить вытягивающий конфиг и ключ — в разделе для сисадминов ниже.

► Полный diff моего `btrbk.conf` против примера

Перед боевым запуском полезно прогнать «вхолостую»:

```
btrbk -c /etc/btrbk/btrbk.conf dryrun
btrbk -c /etc/btrbk/btrbk.conf run
```

Объяснить с

Под капотом: `btrfs send / receive`

Инкрементальность держится на паре `btrfs send / btrfs receive`. `send` умеет выгрузить дамп подтома в любой поток вывода. А если дать ему два снимка — он выгрузит разницу между ними:

```
# Полный поток первого снимка
btrfs send /home/.snapshots/home.20260514 | btrfs receive /mnt/backup/home/

# Только дельта между двумя снимками
btrfs send -p /home/.snapshots/home.20260514 /home/.snapshots/home.20260515 | btrfs
receive /mnt/backup/home/
```

Объяснить с

Первый прогон делает базовый снимок и передаёт его целиком. Каждый следующий прогон делает новый снимок, вычисляет разницу с предыдущим и отправляет на приёмник только изменения. Приёмник — тоже `Btrfs`, локальный или удалённый. И поскольку это поток на `stdout/stdin`, его можно прогнать через `ssh`, `pgp`, `zstd` — что угодно. `btrbk` всю эту механику (поиск общего родителя, инкремент, ротацию) берёт на себя.

Интересный момент: принимающая ФС может быть меньше по размеру, чем отдающая. Главное, чтобы на ней нашлось место для снимка.

Раздел для сисадминов: инкрементальный бакап по SSH за минуты

Удалённая копия делается с бакап-сервера: он сам ходит на рабочий сервер по SSH и забирает готовые снимки. На рабочем сервере свой `btrbk run` уже создал снимки и положил локальную копию (копии 1 и 2) — бакап-серверу остаётся вытянуть инкремент к себе (копия 3).

Конфиг `/etc/btrbk/btrbk.conf` на бакап-сервере — источник удалённый, приёмник локальный:

```
# На БЭКАП-сервере: вытягиваем готовые снимки с рабочего сервера
ssh_identity      /etc/btrbk/ssh/id_ed25519
backend_remote    btrfs-progs-sudo

volume ssh://prod.example.com/
  snapshot_dir      .snapshots
  snapshot_create    no          # снимки уже сделал btrbk на рабочем сервере
  snapshot_preserve_min all
  target_preserve    7d 4w 3m 1y
  subvolume root
    target /mnt/btr_backup/prod/root
  subvolume home
    target /mnt/btr_backup/prod/home
```

Объяснить с

Почему вытягивает данные бакап-сервер, а не наоборот

Это вопрос безопасности, а не вкуса. Ключ и доступ к рабочему серверу держит бакап-сервер; сам рабочий сервер про хранилище не знает ничего и доступа к нему не имеет. Если рабочий сервер взломают или он поймает шифровальщик — атакующий физически не дотянется до бакапов: у скомпрометированной машины нет ни ключа, ни маршрута до хранилища. При обратной схеме (рабочий сам толкает копию наружу) ключ к хранилищу лежал бы на самой уязвимой машине и достался бы атакующему вместе с ней. Бакап должен сам приходить за данными в защищаемую систему, а не наоборот.

Это общепринятый принцип безопасного бакапа, например, документация BorgBackup (команда `borg` — популярная опенсорсная программа резервного копирования) в разделе [«How can I protect against a hacked backup client?»](#) прямо рекомендует pull-режим против скомпрометированного клиента. И сам `btrbk` документирует эту схему в `btrbk.conf.example` блоком «Resume backups from remote host which runs its own btrbk instance» — это тот pull-конфиг, что приведён выше.

Безопасная настройка бакапа по SSH

На рабочем сервере не нужен для бакап-сервера ни root, ни шелл. btrbk поставляется со скриптом-фильтром `ssh_filter_btrbk.sh`. Кладём его на рабочий сервер и в `~/.ssh/authorized_keys` ограничиваем ключ бакап-сервера ролью «только отдать снимки» (`--send` — режим «чтение наружу», `-p` — ограничить путём):

```
command="/usr/share/btrbk/scripts/ssh_filter_btrbk.sh -l --send -p /",restrict ssh-ed25519 AAAA...ключ бакап-сервера...
```

Объяснить с

Даже если взломан сам бакап-сервер, его ключ к рабочему серверу умеет только читать снимки указанных путей — ни удалить, ни зашифровать боевые данные он не может. Эта строка `command="..."` — не команда, которую вы где-то набираете: она один раз вписывается в `authorized_keys` на рабочем сервере, и `sshd` сам подставляет её при входе по ключу бакап-сервера.

Технически это SSH forced command (принудительно заданная команда), а не login shell пользователя. Прописывается она в файле `~/.ssh/authorized_keys` бакап-аккаунта **на рабочем (защищаемом) сервере** — на той стороне, к которой бакап-сервер ходит за данными. Зачем: для этого ключа `sshd` игнорирует любую запрошенную клиентом команду и всегда запускает только `ssh_filter_btrbk.sh`; исходную команду фильтр получает в переменной окружения `$SSH_ORIGINAL_COMMAND`, сверяет с белым списком (нужные `btrfs send/receive/subvolume ...` и заданные ключом `-p` пути) и всё лишнее отклоняет. Поэтому неважно, какой у бакап-аккаунта шелл, — ограничение висит на ключе, а не на пользователе. Слово `restrict` в той же строке дополнительно отключает PTY, проброс портов, проброс SSH-агента (`ssh-agent`) и X11.

► Как ограничить права бакап-пользователя на рабочем сервере

Почему бакап занимает минуты, а не часы

По проводам скачивается не весь `/home`, а `btrfs send -p` между вчерашним и сегодняшним снимком — только дельта на уровне блоков. Переименовали каталог на 100 ГБ? Уйдёт несколько килобайт метаданных. Ночной инкремент сервера, где за сутки реально поменялись единицы гигабайт, передаётся за минуты по обычному каналу.

Расписание — два `btrbk run`. На каждой машине свой cron, ночью; бакап-сервер запускается чуть позже, когда снимки на рабочем сервере уже готовы:

```
# на РАБОЧЕМ сервере (свой btrbk.conf) — локальные копии
20 2 * * * root /usr/bin/btrbk run
```

```
# на БЭКАП-сервере (свой btrbk.conf) — вытянуть инкремент с рабочего сервера
40 2 * * * root /usr/bin/btrbk run
Объяснить с
```

Команда одна и та же — `btrbk run`, разные только конфиги на каждой стороне. Руками в любой момент — то же самое (`btrbk dryrun` — холостой прогон). Длинные строки с `ssh_filter_btrbk.sh` нигде набирать не нужно: `btrbk run` вызываете вы, а `command="..."` за вас вызывает `sshd`.

Разовая настройка двух конфигов и одного ключа — и дальше рабочий сервер каждую ночь делает локальные копии, а бэкап-сервер сам приходит за инкрементом и чистит старые копии по политике `7d 4w 3m 1y`. «Правильный бэкап» перестаёт быть сложным проектом и становится всего двумя строчками в `cron`.

Загрузка со снимков: grub-btrfs

Приятный бонус: утилита `grub-btrfs` добавляет в меню GRUB пункты для загрузки с любого read-only снимка. Сломалось обновление системы — перезагружаемся, выбираем в меню вчерашний снимок и грузимся с рабочей системы. Идёт в комплекте с демоном, который сам регенерирует меню GRUB при создании/удалении снимков.

Названия пакета по дистрибутивам:

Дистрибутив	Пакет
Gentoo / Calculate Linux	<code>app-backup/grub-btrfs</code> — в официальном дереве Gentoo пакета нет, он в оверлее GURU (<code>eselect repository enable guru</code>)
Arch Linux	<code>grub-btrfs</code> (репозиторий <code>extra</code>)
Fedora	через COPR-репозиторий <code>grub-btrfs</code> либо из исходников
Debian / Ubuntu	в официальных репозиториях нет — из исходников
openSUSE	штатно использует свою связку <code>snapper</code> + плагин GRUB

Если в вашем дистрибутиве (или, как в Gentoo, в основном репозитории) пакета нет, grub-btrfs можно поставить из исходников. Исходники и инструкция по установке — на странице проекта на GitHub: github.com/Antynea/grub-btrfs. Это `make install` плюс включение демона (`grub-btrfsd`), который сам следит за каталогом снимков и регенерирует меню GRUB.

Связка получается отличная: перед рискованным апдейтом — `create_snapshot.sh before_update`, обновление, при беде — выбор снимка прямо в загрузчике. Откат системы измеряется не часами переустановки, а одной перезагрузкой.

Как восстановиться

Бакап существует только тогда, когда из него можно восстановиться. Поэтому сразу — обратный путь. Команды `btrbk restore` нет, и это сознательно: `btrbk` создаёт и рассылает снимки, а восстановление делается руками на `btrfs send / btrfs receive`. Меньше магии — меньше шансов снести рабочие данные «одной кнопкой». Официальная процедура — в [документации btrbk](#).

Вернуть пару файлов

Снимок — обычный каталог, по нему можно ходить. Копируем нужное обратно:

```
cp -a --reflink=always /home/.snapshots/home.20260514/проект /home/user/
```

Объяснить с

`--reflink=always` — копия мгновенная и без лишнего места (та самая CoW). Самый частый случай, и он тривиален.

Откатить всю ФС к снимку

Это полная замена подтома: мгновенно и атомарно. Два общих условия для обоих способов ниже:

- `/home` (или `/`) должен быть размонтирован, без открытых файлов: всё делаем из `rescue/live-USB` или в однопользовательском режиме, через монтирование с опцией `subvolid=5`. Живую смонтированную ФС так не заменить;
- снимки `btrbk` — `read-only` (иначе `btrfs send` не работает), поэтому живой записываемый подтом получаем снимком с `read-only` снимка (не переключайте `read-only` → `read-write` через `btrfs property set` — берите именно `btrfs subvolume snapshot`);

- /mnt/top в командах ниже — временная точка монтирования: создайте её (mkdir /mnt/top), а после восстановления размонтируйте и удалите. /dev/sdX — заглушка под реальное устройство тома. Если / и /home — разные btrfs, для отката /home подставляйте устройство, на котором он лежит (узнать: findmnt -no SOURCE /home).

► subvol= vs subvolid= и «подтом по умолчанию» — это важно для отката

Способ 1, самый быстрый — set-default. Подтома не двигаем вообще: переключаем подтом по умолчанию. Старый подтом не трогаем — и не упираемся в удаление вложенных .snapshots:

```
mount -o subvolid=5 /dev/sdX /mnt/top
# из хорошего read-only снимка — записываемый подтом
# (read-only корень корректно не загрузится):
btrfs subvolume snapshot /mnt/top/.snapshots/root.20260514 /mnt/top/root.RESTORED
# назначить его подтомом по умолчанию — по пути...
btrfs subvolume set-default /mnt/top/root.RESTORED
# ...или по ID (ID смотрим в `btrfs subvolume list /mnt/top`):
# btrfs subvolume set-default 392 /mnt/top
reboot
Объяснить с
```

Работает, если ФС монтируется через подтом по умолчанию. Если в /etc/fstab или в cmdline ядра жёстко прописан subvol=/subvolid=, set-default игнорируется (по документации он «прячет» верхний подтом и «takes action on next mount») — тогда правьте /etc/fstab/cmdline или используйте способ 2. При выполнении условия это и правда самый быстрый откат всей ФС: ни копирования, ни переноса подтомов, ни возни с home.OLD — мгновенный снимок, смена указателя и одна перезагрузка. По сути так устроен «rollback» в openSUSE/snapper.

Способ 2 — замена подтома через mv. Подходит, только если одна btrfs содержит и /, и /home, причём /home это отдельный подтом. Если же /home — отдельная btrfs, то mv-замена не сработает: используйте Способ 1.

```
mount -o subvolid=5 /dev/sdX /mnt/top
# из хорошего снимка делаем новый записываемый подтом home.NEW
# (снимки /home лежат в /home/.snapshots):
btrfs subvolume snapshot /mnt/top/home/.snapshots/home.20260514 /mnt/top/home.NEW
# отодвигаем испорченный home в home.OLD, ставим home.NEW на его место:
mv /mnt/top/home /mnt/top/home.OLD
mv /mnt/top/home.NEW /mnt/top/home
# проверить, при необходимости поправить fstab, ребут.
# home.OLD одной командой НЕ удалять: внутри /.snapshots — вложенные подтомы
# (история снимков и родители инкрементальной цепочки). См. оговорку ниже.
Объяснить с
```

Важная оговорка про `mv` директории со снимками. Снимки в Btrfs не рекурсивны: новый `/home` (снимок `home.20260514`) получит лишь пустую заглушку `.snapshots`, а вся локальная история снимков и родители инкрементальной цепочки останутся внутри отодвинутого `home.OLD`. Поэтому `home.OLD` нельзя снести одной `btrfs subvolume delete` — он содержит вложенные подтомы; рекурсивное `-R` уничтожит историю и родителей, и следующий бакап придётся делать полным. Из-за этого для полного отката `/home` чище восстановление с бакап-копии (ниже): там принятый снимок самостоятельный, не вложенный. А чтобы локальный откат был тривиальной заменой без этих оговорок, снимки стоит хранить не внутри `/home`, а в отдельном подтоме вне его — тогда история переживает любую замену подтома.

Восстановить с бакап-диска или бакап-сервера

Если рабочий диск цел, но данные испорчены, — тянем снимок с бакапа обратно. На бакапе снимок лежит отдельным самостоятельным подтомом (не вложенным), поэтому для полного отката `/home` он даже удобнее локального. Делаем при размонтированном `/home`, смонтировав по `subvolid=5` том рабочего сервера, на котором лежит `/home`:

```
mount -o subvolid=5 /dev/sdX /mnt/top
# принять снимок с бакап-сервера в верхний уровень рабочего диска
ssh backup-server btrfs send /mnt/btr_backup/prod/home/home.20260514 | btrfs
receive /mnt/top/
# из принятого read-only снимка – живой rw-подтом на месте /home
mv /mnt/top/home /mnt/top/home.OLD
btrfs subvolume snapshot /mnt/top/home.20260514 /mnt/top/home
# поправить fstab/set-default если нужно, проверить, ребут.
# home.OLD – испорченный /home; в нём тоже остаётся /.snapshots,
# поэтому удалять осознанно и рекурсивно (-R), когда бакап уже переснят.
Объяснить с
```

Снимки `home.20260514` и на бакапе, и на рабочем диске держим, пока не пройдёт новый успешный бакап, — иначе рвётся инкрементальная цепочка.

Прогоните это на тестовой машине заранее, пока восстановление учебное, а не боевое.

Заключение

Соберём всё вместе.

- **Btrfs** даёт мгновенные copy-on-write снимки и передачу изменений на уровне блоков и метаданных — переименование каталога на 100 ГБ стоит несколько килобайт трафика, а не 100 ГБ.
- **btrbk** превращает это в автоматический бакап с ротацией: ежедневные, недельные, месячные, годовые копии без вашего участия.
- **Скорость и экономия места.** Само резервное копирование через btrbk крайне быстрое и экономное по диску: передаётся и хранится только блочная дельта, а снимки делят между собой общие данные. На том же объёме помещается в разы больше копий, чем у традиционных средств, — глубокую историю можно держать почти даром.
- **Бакап по SSH** даёт третью, удалённую копию правила 3-2-1, и ночной инкремент рабочего сервера уезжает за минуты.
- **grub-btrfs** превращает откат системы в выбор пункта меню при загрузке.

И помните про «бакап Шрёдингера» — народное правило сисадминов: состояние резервной копии неизвестно, пока не предпринята попытка восстановления. Пока вы не восстановили данные хотя бы раз — у вас не бакап, а надежда на него. Поэтому, настроив всё, поднимите снимок на тестовой машине и убедитесь, что всё восстановилось.

Бакапим всё и всегда — и пусть ваши данные живут ВЕЧНО!

© 2026 ООО «МТ ФИНАНС»